

TRINNITY VISERON SYSTEM

RELATÓRIO DE INCREMENTOS

O que é Viseron • O que o sistema pode fazer • Tudo o que foi incrementado

Ø=ÜQ Pedro Costa — Comandante Supremo
Ø=Üx Trinnity Hurtado — Rainha Arquiteta

Gerado automaticamente em 31/07/2026, 16:15:10

1. O QUE É VISERON

Viseron é o coração do Trinnity Viseron System v5.0: um Sistema Operacional Multi-Agente de IA (Multi-Agent AI Operating System) desenhado para operar como uma superinteligência autônoma. Não é um chatbot: é uma organização de IA com hierarquia, memória, evolução e orquestração próprias, capaz de planejar, executar e aprender por conta própria.

1.1 — Hierarquia de comando

- Ø=ÜQ Pedro Costa — Supreme Commander & Criador do TVS. Comanda a operação e toma as decisões de topo.
- Ø=Üx Trinnity Hurtado — Queen & Chief Architect. Desenha a arquitetura e assina toda decisão com a coroa.

1.2 — Organização interna

- SuperMind: 500 anos de conhecimento acumulado (eras primordial !' futurista), alimentado por 5.000 mentes.
- 246+ arquetipos de agentes e um batallón de 114 agentes operacionais (linhagens Corona e Hierro).
- Total de agentes no arranque: 5.370 (5.000 mentes + 246 arquetipos + 114 batallón + núcleo).

1.3 — Filosofia operacional

- IA local (Ollama) como padrão — sem custo e com privacidade; nuvem só para raciocínio complexo.
- Testes antes do deploy, documentação de cada mudança e proteção do núcleo (regras do AGENTS.md).
- Evolução contínua: cada ciclo de aprendizado aumenta a inteligência dos agentes (+5% com teto seguro).

2. O QUE O SISTEMA PODE FAZER

2.1 — Inteligência e raciocínio

- SuperIntelligence: sintetiza respostas por ensemble de 5.000 mentes para perguntas complexas.
- ModelRouter: escolhe entre 11 providers (Ollama local, OpenAI, Claude, Gemini, Grok) conforme privacidade e qualidade exigidas.
- OmniRoute: gateway opcional com 290+ provedores de IA.

2.2 — Autonomia e orquestração

- Orquestrador de tarefas: divide objetivos em subtarefas e distribui para agentes especializados.
- Squads: equipes de agentes com líderes (Executive, Core Engineering, AIOX) que executam em paralelo.
- Agentes CEO/Subagentes: planejamento autônomo com execução e revisão de código.

2.3 — Memória e aprendizado

- Memória de curto prazo (STM) por sessão e memória de longo prazo (LTM) persistida em database/memory/ltn.json.
- HyperLearning: ciclos de auto-evolução que melhoram o conhecimento e a inteligência dos agentes.
- Qdrant (opcional): memória vetorial semântica; sem ele, usa fallback em RAM.

2.4 — Geração de produtos

- WebAppGenerator: cria sites e apps completos (HTML + mobile) com tokens, páginas e identidade visual.
- Tokenomics: gera tokens ERC-20 / BEP-20 / Solana.
- Forge (Git próprio): hospeda e faz push/pull de repositórios git internos.

2.5 — Interfaces e comunicação

- WebOS Dashboard (<http://localhost:3000>): monitoramento em tempo real, estatísticas, voz e chat.
- Servidor de relatórios PDF (porta 3001): relatórios operacionais e abrangentes.
- Voz: comandos por microfone, VoiceBridge + Socket.IO em tempo real.
- OpenJarvis (IA local) e ASNO (WhatsApp + casa inteligente) e CallSystem (chamadas via Twilio).
- App móvel Expo (Android APK / iOS IPA) que conecta ao servidor do sistema.

2.6 — Integrações e automação

- n8n (automação de workflows), Home Assistant (dispositivos), MCP (Model Context Protocol).

- Mercado / launch: ferramentas de lançamento com análise de mercado.

2.7 — Deploy e operação

- Deploy automático: GitHub, Vercel (site), Render (API), Hostalia (FTP).
- Backups diários com agendador do Windows e proteção de segredos (.env fora dos backups).
- Construção de executáveis standalone (.exe/.app) e aplicativo Electron.

3. INCREMENTOS — SESSÃO 1: BARRIDO E CORREÇÕES

Uma varredura completa orquestrada pelos squads AIOX (Core, Integrações, Deploy, Inteligência) encontrou e corrigiu ~40 falhas. Principais correções:

Área	Falha encontrada	Correção aplicada
Dashboard WebOS	webos.js com erro de sintaxe (interface morta)	Sintaxe corrigida — dashboard volta a funcionar
Forge Git	Rotas Express 5 inválidas (crash no arranque) + git push vazio	Rotas corrigidas + git-receive-pack/upload-pack reais
Configuração	.env nunca era carregado (IA cloud toda em modo simulado)	dotenv/config carregado nos servidores
AgentSpawner	Caminho de minds.json quebrado em modo dev	Resolução robusta (cwd + dirname)
IA	4.756 mentes usavam provider 'openai' (violava a regra Ollama)	Provider padrão agora é ollama (local)
Scripts npm	call:start, jarvis:start, asno:start, omniroute:start quebrados	startServer exportado + guarda contra null
WebAppGenerator	Sites token com sintaxe inválida + portas 3000 colidindo	Interpolação corrigida + portas 4100+
HyperLearning	Inteligência explodia x6 por ciclo (valor sem sentido)	Crescimento +5% com teto de 1.000.000
tvstools	Comando 'transcrever' nunca funcionava (precedência)	Parênteses corrigidos
Produção	Porta 3000 fixa ignorava PORT (crash em Railway/Render)	Agora respeita process.env.PORT
Segurança	.env ia para Docker/pkg/backups; senha n8n fixa	.dockerignore criado; .env fora de pkg/backups; senha via env
Mentes	Missão de 5.000+ mentes não era cumprida (4.756)	Regeneradas: 5.000 mentes (342 históricas + 4.658 sintéticas)
Mobile	Servidor do app fixo não configurável	DEFAULT_SERVER via Expo config (app.json !' extra.tvServerUrl)
Relatórios	providerCount/intelligenceLevel com valores incorretos	Valores reais calculados a partir do sistema

Entregáveis da Sessão 1:

- Manual de operação humana: data/reports/TVS_Pasos_Humanos_Operacion.pdf (12 seções + diagrama).
- Diagrama de operação: data/reports/TVS_Diagrama_Operacao.svg / .png.
- Script regenerável do manual: scripts/gen-manual-operacion.js.

4. INCREMENTOS — SESSÃO 2: BIBLIOTECA DE SKILLS (958 skills)

Integração autônoma de 4 coleções públicas de Agent Skills (formato padrão SKILL.md) como módulo nativo de Viseron. O conteúdo original é preservado em skills/vendor/ com sua licença e autor; toda a camada de integração é de autoria Viseron.

4.1 — Coleções integradas

Coleção	Origem	Licença	Skills
awesome-claude-skills	ComposioHQ/awesome-claude-skills	Apache-2.0	864
superpowers	obra/superpowers	MIT	14

claude-plugins-official	anthropics/claude-plugins-official	Apache-2.0	31
marketingskills	coreyhaines31/marketingskills	MIT	49

Total: 958 skills em 21,6 MB (skills/vendor/ fica no .gitignore e é instalável com um comando).

4.2 — O que a Viseron Skills Library oferece

- Desenvolvimento: test-driven-development, brainstorming, subagent-driven-development, systematic-debugging, git-worktrees.
- Marketing: CRO, copywriting, SEO (seo-audit, programmatic-seo, schema), cold-email, analytics, pricing, launch.
- Documentos e mídia: docx, pdf, pptx, xlsx, canvas-design, image-enhancer, theme-factory, slack-gif-creator.
- Dados e análise: deep-research, csv-summarizer, langsmith-fetch, developer-growth-analysis.
- Automação de apps: GitHub, Slack, Gmail, Notion, Jira, Salesforce, WhatsApp e dezenas de outros (Rube MCP).

4.3 — Módulo de integração (autoría Viseron)

- src/core/skills/SkillsRegistry.ts — escâner de SKILL.md, parser de frontmatter, busca e carga sob demanda (~100 tokens por skill no índice).
- scripts/skills.ts — CLI autônomo com Commander: install, list, search, info.
- API REST no dashboard: GET /api/skills, GET /api/skills/stats, GET /api/skills/:id.
- Scripts npm: skills:install, skills:list, skills:search, skills:info, skills.
- Documentação: docs/SKILLS.md + AGENTS.md atualizado.
- Testes: 4 novos testes de SkillsRegistry no suite (14/14 total).

5. ESTADO VERIFICADO

Verificação	Resultado
TypeScript / lint (tsc --noEmit)	OK, sem erros
Build (compilação + copia de assets)	OK
Testes automatizados	14/14 PASS
Instalação de skills	958 skills / 4 coleções
Busca e leitura de skills	OK (ex.: search 'cro' ! 26 resultados)
API REST de skills	OK (stats, list, detail)
Arranque do sistema	5.370 agentes (5.000 mentes + 246 arquetipos + 114 batallón)
Segurança	.env fora de backups, Docker e executáveis

6. COMANDOS NOVOS (Sessão 2)

O que fazer	Comando
Instalar/atualizar as coleções de skills	npm run skills:install
Listar todas as skills	npm run skills:list
Buscar skills por texto	npm run skills -- search <termo>
Ver conteúdo de uma skill	npm run skills -- info <id>
Filtrar por coleção	npm run skills -- list --source superpowers
Ver skills pelo dashboard	http://localhost:3000/api/skills/stats
Gerar este relatório	node scripts/gen-report-incrementos.js